



UNIVERSIDAD NACIONAL DE COLOMBIA

FACULTAD DE CIENCIAS EXACTAS Y NATURALES

Análisis y Diseño de Algoritmos

Estudio experimental comparativo de algoritmos para el problema del Camino Hamiltoniano

Autores

Juan Diego Ramírez Sánchez
juaramirezs@unal.edu.co

Juan Esteban Agudelo Burgos
juagudelobu@unal.edu.co

Profesor

Jairo Hernán Aponte Melo

Manizales, 15 de julio de 2026

Resumen

El problema del Camino Hamiltoniano —decidir si un grafo admite un recorrido que visite cada vértice exactamente una vez— es un problema NP-completo para el cual no se conoce ningún algoritmo eficiente en el peor caso. En este trabajo implementamos, validamos y comparamos experimentalmente cinco enfoques para resolverlo: cuatro exactos —fuerza bruta, *backtracking*, el procedimiento de búsqueda de Rubin (*backtracking* con reglas de deducción) y programación dinámica (algoritmo de Held–Karp)— y uno probabilístico de tipo Monte Carlo (construcción aleatoria con reinicios). Tras verificar la corrección de cada implementación con una batería de pruebas unitarias, medimos su tiempo de ejecución, su número de operaciones básicas y su uso de memoria sobre instancias de tamaño creciente en tres escenarios (mejor, peor y promedio), ejecutando todo el barrido en un entorno local controlado para obtener mediciones estables y reproducibles.

Los resultados confirman con nitidez el muro de complejidad de los métodos exactos y, a la vez, revelan diferencias prácticas importantes entre ellos. La fuerza bruta fue la primera en volverse impracticable, con un crecimiento del orden de $n \cdot n!$; la programación dinámica resultó la más resistente en tiempo entre los exactos, aunque a costa de un consumo de memoria exponencial que terminó limitándola por espacio y no por tiempo. El *backtracking* y el procedimiento de Rubin escalaron mucho más lejos en instancias favorables gracias a sus podas, pero en el peor caso —grafos sin camino hamiltoniano— esa ventaja se desvanece y su desempeño se acerca al de la fuerza bruta. El algoritmo de Monte Carlo alcanzó los mayores tamaños con un costo polinómico por intento, a cambio de un error de un solo lado: puede no encontrar un camino que sí existe, sobre todo en grafos poco densos. En conjunto, las curvas empíricas de tiempo se ajustaron bien a las cotas teóricas de cada algoritmo, y el estudio ilustra con datos el clásico compromiso entre exactitud, tiempo y memoria en los problemas NP-completos.

Índice

1. Introducción	3
2. Diseño Experimental	4
2.1. Algoritmos evaluados	4
2.1.1. Fuerza bruta	4
2.1.2. Backtracking	5
2.1.3. Procedimiento de búsqueda de Rubin	6
2.1.4. Programación dinámica (algoritmo de Held–Karp)	8
2.1.5. Algoritmo probabilístico de Monte Carlo	9
2.2. Métricas de evaluación	10
2.3. Generación y selección de instancias	11
2.4. Entorno experimental	12
2.5. Procedimiento de experimentación	13
3. Resultados	14
3.1. Comparación del número de operaciones básicas	14
3.2. Tiempo de ejecución frente al tamaño de entrada	14
3.3. Variabilidad entre réplicas	16
3.4. Contraste con la complejidad teórica	17
4. Análisis y Discusión	19
4.1. Interpretación de los patrones observados	19
4.2. Resultados inesperados	19
4.3. Limitaciones y amenazas a la validez	20
4.4. Preguntas abiertas y trabajo futuro	20
5. Conclusiones	21
6. Referencias	22
7. Anexos	22

1. Introducción

Muchos problemas de gran interés teórico y práctico pertenecen a la clase de los problemas **NP-completos**, para los cuales no se conoce ningún algoritmo que los resuelva en tiempo polinomial en el peor caso. Uno de los más representativos es el problema del **Camino Hamiltoniano** (abreviado HAM-PATH), que consiste en decidir si un grafo no dirigido $G = (V, E)$ admite un *camino hamiltoniano*: un recorrido que visite cada uno de sus $n = |V|$ vértices exactamente una vez, pasando siempre por aristas del grafo. Su versión de decisión es NP-completa: pertenece a la clase **NP**, pues un camino candidato puede *verificarse* en tiempo polinomial $O(n)$, y es **NP-difícil**, lo que se establece mediante la reducción $3\text{-SAT} \leq_p \text{HAM-PATH}$. En términos prácticos, esto significa que resolverlo de forma exacta obliga, hasta donde se sabe, a explorar un espacio de soluciones que crece de manera explosiva con el tamaño de la entrada.

En este trabajo estudiamos cinco enfoques para atacar el problema, que van desde la búsqueda exhaustiva hasta la aproximación probabilística. Los cuatro primeros son **exactos**: la *fuerza bruta*, que enumera todas las ordenaciones posibles de los vértices y comprueba cuáles forman un camino; el *backtracking*, que construye el camino de forma incremental y retrocede en cuanto una rama se vuelve inviable; el *procedimiento de búsqueda de Rubin*, que refina el *backtracking* con reglas de deducción capaces de descartar por anticipado los prefijos sin salida; y la *programación dinámica* mediante el algoritmo de *Held–Karp*, que memoriza subproblemas usando máscaras de bits para pasar de un costo factorial a uno exponencial. El quinto enfoque es un algoritmo probabilístico de *Monte Carlo*, que construye caminos al azar con reinicios y acepta equivocarse con cierta probabilidad —un error de un solo lado— a cambio de un costo mucho menor por intento. La Tabla 1 resume las complejidades de peor caso que dedujimos para cada uno, y que se analizan en detalle más adelante.

Tabla 1: Complejidades del peor caso de los cinco enfoques evaluados. Aquí $n = |V|$, $m = |E|$ y k es el número (fijo) de intentos del algoritmo de Monte Carlo.

Algoritmo	Tipo	Tiempo (peor caso)	Espacio (peor caso)
Fuerza bruta	Exacto	$O(n \cdot n!)$	$O(n)$
Backtracking	Exacto	$O(n \cdot n!)$	$O(n)$
Procedimiento de Rubin	Exacto	$O((n + m) \cdot n!)$	$O(n)$
Programación dinámica (Held–Karp)	Exacto	$O(2^n n^2)$	$O(2^n n)$
Monte Carlo	Probabilístico	$O(k(n + m))$	$O(n)$

El resto del informe se organiza de la siguiente manera. En la sección de **Diseño Experimental** describimos con detalle cómo se llevó a cabo el estudio de forma controlada, rigurosa y reproducible: presentamos el pseudocódigo y el análisis de complejidad temporal y espacial de los *algoritmos evaluados*, las *métricas de evaluación* con que monitoreamos las variables dependientes, el procedimiento de *generación y selección de instancias*, el *entorno experimental* de hardware y software donde tomamos las mediciones, y el *procedimiento de experimentación* junto con la estructura de archivos y scripts que permite replicar el trabajo. Enseguida, en **Resultados**, presentamos los resúmenes estadísticos, las tablas comparativas y las figuras que sintetizan el comportamiento observado según el tamaño y el tipo de las instancias. En **Análisis y Discusión** interpretamos esos resultados, los contrastamos con las complejidades teóricas de la Tabla 1, explicamos las diferencias de rendimiento entre los algoritmos y reflexionamos de forma crítica sobre las limitaciones del experimento. Por último, en **Conclusiones** recogemos los hallazgos clave y las lecciones aprendidas —incluido el papel que tuvo el uso de inteligencia artificial en el desarrollo del trabajo— y cerramos con las **Referencias** y los **Anexos**, donde se enlazan los notebooks y los datos del experimento.

2. Diseño Experimental

El diseño experimental es el conjunto de decisiones metodológicas que definen cómo se llevó a cabo el experimento de forma controlada, rigurosa y reproducible. En esta sección detallamos los algoritmos que evaluamos, las métricas con que medimos su desempeño, la manera en que generamos las instancias del problema, el entorno de ejecución donde tomamos las mediciones y el procedimiento exacto que garantizó condiciones comparables entre corridas. El propósito es que cualquier persona pueda repetir el estudio y obtener conclusiones consistentes con las nuestras.

2.1. Algoritmos evaluados

Evaluamos los cinco enfoques presentados en la introducción. Todos comparten la misma interfaz: reciben un grafo no dirigido —un objeto `Graph` de la librería `networkx`— y retornan una pareja (P, R) , donde P es la lista de vértices del camino hamiltoniano hallado (o `NONE` si no existe) y R es el valor booleano de la versión de decisión, cumpliéndose siempre la coherencia $R \equiv (P \neq \text{NONE})$. Esta interfaz uniforme fue clave para poder intercambiar los algoritmos dentro del mismo motor experimental. A continuación presentamos, para cada uno, su idea general, su pseudocódigo y el análisis de su complejidad temporal y espacial del peor caso.

2.1.1. Fuerza bruta

Este primer enfoque resuelve el problema mediante una **búsqueda exhaustiva**. Como un camino hamiltoniano no es más que una ordenación de los n vértices que respeta las adyacencias, basta con enumerar *todas* las permutaciones de V y comprobar, una por una, si alguna constituye un camino válido. Las condiciones de cubrir todos los vértices sin repetirlos se satisfacen de forma automática por tratarse de permutaciones; el único predicado que resta verificar es que cada par de vértices consecutivos sea adyacente.

Algoritmo 1 Fuerza bruta para el problema del Camino Hamiltoniano.

Entrada: Grafo no dirigido $G = (V, E)$, con $n = |V|$.

Salida: Pareja (P, R) : camino P (o `NONE`) y decisión R .

```
1: función FUERZABRUTA( $G$ )
2:    $V \leftarrow$  lista de vértices de  $G$ 
3:   si  $n = 0$  entonces
4:     retornar (NONE, FALSO)
5:   fin si
6:   para cada permutación  $\pi = \langle v_1, \dots, v_n \rangle$  de  $V$  hacer
7:     si  $\{v_i, v_{i+1}\} \in E$  para todo  $i \in \{1, \dots, n-1\}$  entonces
8:       retornar ( $\pi$ , VERDADERO)
9:     fin si
10:  fin para
11:  retornar (NONE, FALSO)
12: fin función
```

Complejidad. El espacio de búsqueda son las $n!$ permutaciones de V ; para cada una, comprobar la condición de adyacencia recorre los $n-1$ pares consecutivos con consultas $O(1)$ (la adyacencia se almacena como un diccionario de diccionarios). En el peor caso —cuando el grafo no admite camino, o cuando el único válido es el último examinado— se recorren todas las permutaciones, de modo que el tiempo es

$$T(n) = O(n \cdot n!).$$

Por la aproximación de Stirling, $n! \approx \sqrt{2\pi n} (n/e)^n$, este crecimiento es *superexponencial*, incluso más costoso que $O(2^n)$. En cuanto al espacio, las permutaciones se generan de forma perezosa (una a la vez), por lo que solo se mantienen en memoria la lista de vértices y la permutación en curso, ambas de tamaño $O(n)$; así, $S(n) = O(n)$.

2.1.2. Backtracking

El *backtracking* refina la búsqueda exhaustiva introduciendo **podar**. En lugar de generar cada permutación completa para recién comprobarla, construye el camino de forma incremental y *abandona de inmediato* cualquier camino parcial que ya no pueda prolongarse hacia un vecino no visitado. Explora así un árbol de decisión cuyos nodos son caminos parciales: descender un nivel equivale a extender el camino hacia un vecino admisible, y retroceder ocurre cuando se llega a un callejón sin salida. Como el problema no fija un vértice de partida, se prueba cada vértice como posible inicio.

Algoritmo 2 Backtracking para el problema del Camino Hamiltoniano.

Entrada: Grafo no dirigido $G = (V, E)$, con $n = |V|$.

Salida: Pareja (P, R) .

```

1: función BACKTRACKING( $G$ )
2:   si  $n = 0$  entonces
3:     retornar (NONE, FALSO)
4:   fin si
5:   para cada vértice  $s \in V$  hacer
6:      $P \leftarrow \langle s \rangle$ ;  $vis \leftarrow \{s\}$ 
7:     si EXTENDER( $P, vis$ ) entonces
8:       retornar ( $P$ , VERDADERO)
9:     fin si
10:  fin para
11:  retornar (NONE, FALSO)
12: fin función

13: función EXTENDER( $P, vis$ )
14:  si  $|P| = n$  entonces
15:    retornar VERDADERO
16:  fin si
17:   $u \leftarrow$  último vértice de  $P$ 
18:  para cada  $w \in$  vecinos( $u$ ) con  $w \notin vis$  hacer
19:    agregar  $w$  a  $P$ ; agregar  $w$  a  $vis$ 
20:    si EXTENDER( $P, vis$ ) entonces
21:      retornar VERDADERO
22:    fin si
23:    quitar  $w$  de  $P$ ; quitar  $w$  de  $vis$  ▷ retroceso
24:  fin para
25:  retornar FALSO
26: fin función

```

Complejidad. En el peor caso —un grafo completo K_n , donde ninguna rama llega a podarse— el algoritmo recorre, sobre los n posibles vértices de inicio, del orden de $n!$ caminos parciales, con un costo $O(n)$ por nodo del árbol de recursión; por tanto

$$T(n) = O(n \cdot n!),$$

la misma cota asintótica que la fuerza bruta. No obstante, esta cota es *pesimista*: en grafos dispersos o sin camino hamiltoniano, la poda descarta caminos parciales inviables de forma temprana y el número de nodos realmente explorados es muchísimo menor, lo que constituye la ventaja práctica —no asintótica— del *backtracking* sobre la fuerza bruta. El espacio está dominado por la pila de recursión, de profundidad máxima n , más el conjunto de visitados y el camino parcial, todos de tamaño $O(n)$; así, $S(n) = O(n)$.

2.1.3. Procedimiento de búsqueda de Rubin

Frank Rubin (1974) propuso un *backtracking* **inteligente** que, antes de extender un camino parcial, aplica un conjunto de **reglas de deducción** para detectar de forma anticipada si ese prefijo es un callejón sin salida —es decir, si ningún camino hamiltoniano puede contenerlo— y podar toda la rama. De su repertorio de reglas implementamos un subconjunto correcto y de alto poder de poda, adaptado a grafos no dirigidos. Sea P el camino parcial, u su último vértice y $Pend$ los vértices no visitados; los únicos que aún pueden formar nuevas aristas del camino son u y los de $Pend$, conjunto que llamamos $Disp$. Sobre el subgrafo inducido por $Disp$ aplicamos: **(F1)** si algún pendiente queda aislado, es inadmisibile; **(F5)** a lo sumo un pendiente puede tener grado 1 (sería el otro extremo del camino); y **(F7)** todos los pendientes deben ser alcanzables desde u .

Algoritmo 3 Procedimiento de Rubin: rutina principal.

Entrada: Grafo no dirigido $G = (V, E)$, con $n = |V|$.

Salida: Pareja (P, R) .

```

1: función RUBIN( $G$ )
2:   si  $n = 0$  entonces
3:     retornar (NIL, FALSO)
4:   fin si
5:   para cada vértice  $s \in V$  hacer
6:      $P \leftarrow \langle s \rangle$ ;  $vis \leftarrow \{s\}$ 
7:     si BUSCAR( $P, vis$ ) entonces
8:       retornar ( $P$ , VERDADERO)
9:     fin si
10:  fin para
11:  retornar (NIL, FALSO)
12: fin función
```

Algoritmo 4 Procedimiento de Rubin: extensión con poda (BUSCAR).

```
1: función BUSCAR( $P, vis$ )
2:   si  $|P| = n$  entonces
3:     retornar VERDADERO
4:   fin si
5:   si no ADMISIBLE( $P, vis$ ) entonces
6:     retornar FALSO
7:   fin si ▷ poda por deducción
8:    $u \leftarrow$  último vértice de  $P$ 
9:   para cada  $w \in \text{vecinos}(u)$  con  $w \notin vis$  hacer
10:    agregar  $w$  a  $P$  y a  $vis$ 
11:    si BUSCAR( $P, vis$ ) entonces
12:      retornar VERDADERO
13:    fin si
14:    quitar  $w$  de  $P$  y de  $vis$ 
15:  fin para
16:  retornar FALSO
17: fin función
```

Algoritmo 5 Procedimiento de Rubin: test de admisibilidad (ADMISIBLE).

```
1: función ADMISIBLE( $P, vis$ )
2:    $u \leftarrow$  último de  $P$ ;  $Pend \leftarrow V \setminus vis$ 
3:   si  $Pend = \emptyset$  entonces
4:     retornar VERDADERO
5:   fin si
6:    $Disp \leftarrow Pend \cup \{u\}$ ;  $g_1 \leftarrow 0$ 
7:   para cada  $x \in Pend$  hacer
8:      $d \leftarrow |\{y \in Disp : \{x, y\} \in E\}|$  ▷ grado en el subgrafo  $Disp$ 
9:     si  $d = 0$  entonces
10:      retornar FALSO
11:     fin si ▷ (F1) vértice aislado
12:     si  $d = 1$  entonces
13:        $g_1 \leftarrow g_1 + 1$ 
14:       si  $g_1 \geq 2$  entonces
15:         retornar FALSO
16:       fin si ▷ (F5) dos extremos
17:     fin si
18:   fin para
19:   si algún vértice de  $Pend$  no es alcanzable desde  $u$  dentro de  $Disp$  entonces
20:     retornar FALSO ▷ (F7) conectividad
21:   fin si
22:   retornar VERDADERO
23: fin función
```

Complejidad. En el peor caso —un grafo completo, donde las reglas nunca podan— la búsqueda degenera en el *backtracking* exhaustivo, recorriendo del orden de $n!$ caminos parciales; la diferencia es que cada nodo de la recursión paga ahora el costo del test de admisibilidad, dominado por el recorrido de conectividad (F7), que es $O(n + m)$. Por tanto

$$T(n) = O((n + m) \cdot n!).$$

Al igual que en el *backtracking*, esta es una cota pesimista: las reglas de deducción reducen drásticamente el árbol de búsqueda en instancias reales, aunque a costa de un mayor trabajo por nodo. El espacio lo dominan la pila de recursión y las estructuras auxiliares del test (todas de tamaño $O(n)$, reutilizadas en cada llamada); así, $S(n) = O(n)$.

2.1.4. Programación dinámica (algoritmo de Held–Karp)

La programación dinámica ataca la principal ineficiencia de los enfoques anteriores: el recálculo de situaciones equivalentes. La observación clave es que, para decidir cómo continuar un camino parcial, solo importan dos datos: *qué* conjunto de vértices se ha visitado y *en cuál* termina; el orden exacto de la visita es irrelevante. Como hay hasta $n!$ órdenes pero solo $2^n \cdot n$ situaciones distintas, memorizarlas produce una mejora enorme. Definimos el estado $dp[S][v] = \text{VERDADERO}$ si existe un camino simple que visita *exactamente* el conjunto $S \subseteq V$ y termina en v . Los subconjuntos S se representan como **máscaras de bits** (un entero cuyo bit i indica si el vértice i pertenece a S), lo que vuelve las operaciones de conjunto simples operaciones de bits en tiempo $O(1)$.

Algoritmo 6 Programación dinámica (Held–Karp) para el Camino Hamiltoniano.

Entrada: Grafo no dirigido $G = (V, E)$, con $n = |V|$ y vértices indexados $0, \dots, n - 1$.

Salida: Pareja (P, R) .

```

1: función PROG DINÁMICA( $G$ )
2:   si  $n = 0$  entonces
3:     retornar (NIL, FALSO)
4:   fin si
5:    $dp[S][v] \leftarrow \text{FALSO}$  para todo  $S \subseteq V, v \in V$ 
6:   para cada  $v \in V$  hacer
7:      $dp[\{v\}][v] \leftarrow \text{VERDADERO}$  ▷ caso base: camino de un vértice
8:   fin para
9:   para cada subconjunto  $S \subseteq V$  en orden creciente de máscara hacer
10:    para cada  $v \in S$  con  $dp[S][v] = \text{VERDADERO}$  hacer
11:      para cada  $w \in \text{vecinos}(v)$  con  $w \notin S$  hacer
12:         $dp[S \cup \{w\}][w] \leftarrow \text{VERDADERO}$ 
13:         $padre[S \cup \{w\}][w] \leftarrow v$ 
14:      fin para
15:    fin para
16:   fin para
17:   para cada  $v \in V$  hacer
18:     si  $dp[V][v] = \text{VERDADERO}$  entonces
19:       reconstruir  $P$  siguiendo padre desde  $(V, v)$  hacia atrás
20:     retornar ( $P$ , VERDADERO)
21:   fin si
22: fin para
23: retornar (NIL, FALSO)
24: fin función
```

Complejidad. El número de estados es $2^n \cdot n$ (subconjuntos por vértice final). Cada estado alcanzable examina los vecinos de v —a lo sumo $n - 1$ — con trabajo $O(1)$ por vecino, de modo que el trabajo por estado es $O(n)$. En consecuencia,

$$T(n) = O(2^n n^2),$$

una mejora enorme frente al $O(n!)$ de los enfoques de búsqueda. La contrapartida está en el espacio: las tablas dp y $padre$ tienen $2^n \cdot n$ entradas cada una, por lo que

$$S(n) = O(2^n n).$$

A diferencia del *backtracking*, Held–Karp *intercambia espacio por tiempo*: su memoria exponencial es, en la práctica, lo que limita su aplicabilidad, no el tiempo.

2.1.5. Algoritmo probabilístico de Monte Carlo

A diferencia de los cuatro enfoques anteriores —deterministas y exactos—, este algoritmo usa el azar y puede equivocarse con cierta probabilidad, a cambio de un costo mucho menor. Construye caminos al azar mediante caminatas sin repetición: parte de un vértice aleatorio y, en cada paso, salta a un vecino no visitado elegido uniformemente al azar; si completa los n vértices, devuelve el camino, y si se atasca, reinicia con otro intento. Tiene un **error de un solo lado**: nunca produce un falso positivo (todo camino que reporta es real), pero puede fallar por falso negativo —responder que no existe cuando sí existía, sin que el azar lo hallara—.

Algoritmo 7 Monte Carlo (construcción aleatoria con reinicios).

Entrada: Grafo no dirigido $G = (V, E)$ con $n = |V|$, número de intentos T .

Salida: Pareja (P, R) .

```

1: función MONTECARLO( $G, T$ )
2:   si  $n = 0$  entonces
3:     retornar (NIL, FALSO)
4:   fin si
5:   para  $i \leftarrow 1, \dots, T$  hacer
6:      $u \leftarrow$  vértice aleatorio de  $V$ 
7:      $P \leftarrow \langle u \rangle$ ;  $vis \leftarrow \{u\}$ 
8:     mientras  $|P| < n$  hacer
9:        $C \leftarrow \{w \in \text{vecinos}(u) : w \notin vis\}$ 
10:      si  $C = \emptyset$  entonces
11:        salir del ciclo ▷ atasco
12:      fin si
13:       $u \leftarrow$  elemento aleatorio de  $C$ 
14:      agregar  $u$  a  $P$  y a  $vis$ 
15:    fin mientras
16:    si  $|P| = n$  entonces
17:      retornar ( $P$ , VERDADERO)
18:    fin si
19:  fin para
20:  retornar (NIL, FALSO)
21: fin función

```

Complejidad. Cada intento realiza a lo sumo n pasos y, en cada paso, listar los vecinos no visitados cuesta $O(g(v))$ siendo v el vértice actual para el intento y $g(v)$ el grado de v ; sumado, un intento cuesta $O(n + m)$. Con k intentos,

$$T(n) = O(k(n + m)),$$

que es *polinomial* para k fijo. La dificultad no está en el costo por intento, sino en cuántos intentos hacen falta para una probabilidad de éxito alta: si el grafo tiene camino, cada intento lo halla con probabilidad $q > 0$, y la probabilidad de fallar en los k intentos es $(1 - q)^k$; pero q

depende de la estructura del grafo y puede ser diminuto en grafos poco densos, lo que revela la naturaleza heurística del método. El espacio es $S(n) = O(n)$, pues solo se almacenan el camino y el conjunto de visitados.

2.2. Métricas de evaluación

En nuestro experimento controlamos tres **variables independientes** —el algoritmo, el tamaño de la entrada n y el tipo de caso (mejor, peor o promedio)— y observamos su efecto sobre tres **variables dependientes**: el tiempo de ejecución, el número de operaciones básicas y el uso de memoria. Para que la comparación entre los cinco algoritmos fuera justa, medimos cada variable con la misma herramienta y bajo el mismo procedimiento para todos ellos. A continuación describimos cada métrica.

Tiempo de ejecución (segundos). Es la métrica principal: el tiempo real que tarda cada algoritmo en resolver una instancia. Lo medimos con `time.perf_counter()`, el reloj de mayor resolución de Python y el recomendado para *benchmarking*, encapsulado —siguiendo la guía de Real Python— en una clase `Timer` que extendimos con el protocolo de *context manager* (`__enter__`/`__exit__`), de modo que cronometrar un bloque se reduce a `with Timer() as t: ...`. Esta encapsulación garantiza que el mecanismo de medición sea idéntico para los cinco algoritmos.

Número de operaciones básicas. Complementa al tiempo con una medida *independiente del hardware*. Como operación básica común a los cinco algoritmos adoptamos la **consulta de adyacencia** —comprobar si dos vértices son vecinos o recorrer la vecindad de un vértice—, por ser el primitivo que todos comparten y el que domina su costo. Para contarla *sin modificar* los algoritmos ya validados, envolvimos el grafo en una clase `GrafoContador` que delega en el grafo real pero incrementa un contador en cada consulta; basta ejecutar el algoritmo sobre el grafo envuelto y leer el contador. Conviene precisar que esta métrica mide el *trabajo primitivo total*, incluida la sobrecarga de las podas: por eso un algoritmo que explora menos nodos (como el de Rubin) puede, aun así, realizar más consultas de adyacencia por nodo. Optamos por esta única métrica —en lugar de contar además llamadas recursivas— precisamente porque es comparable entre enfoques de naturaleza distinta (búsqueda, programación dinámica y muestreo aleatorio).

Uso de memoria (kilobytes). Registra el **pico** de memoria asignada durante la ejecución, la variable decisiva para la programación dinámica. La medimos con el módulo `tracemalloc` de la biblioteca estándar, que rastrea las asignaciones de memoria de Python y reporta el máximo alcanzado durante el bloque medido.

Un punto metodológico importante es que estas tres métricas **no** pueden medirse en una misma corrida, porque tanto el contador de operaciones como `tracemalloc` introducen una sobrecarga que *inflaría* el tiempo. Por ello cada medición se toma en una ejecución independiente: una corrida limpia y sin instrumentación para el tiempo, una segunda sobre el grafo envuelto para las operaciones, y una tercera bajo `tracemalloc` para el pico de memoria.

Además de estas tres variables, en cada corrida registramos información auxiliar que enriquece el análisis: el tamaño n y el número de aristas m de la instancia, si el algoritmo *encontró* o no un camino, y la longitud del camino hallado. Cada réplica se almacena como una fila de datos brutos, y a partir de ellas obtenemos después las **medidas de tendencia central** (media, mediana y moda) y de **dispersión** (desviación estándar, mínimo y máximo) que se presentan en la sección de Resultados.

2.3. Generación y selección de instancias

Las instancias del problema son grafos no dirigidos que generamos de forma **controlada y reproducible**, variando dos parámetros: el tamaño n (número de vértices) y el *tipo de caso*. Definimos tres tipos de caso, que corresponden al análisis clásico de mejor, peor y promedio, y que buscan cubrir todo el espectro de dificultad del problema:

Mejor caso —grafo completo K_n . Todo par de vértices es adyacente, de modo que cualquier ordenación de los vértices es un camino hamiltoniano. Los algoritmos de búsqueda lo encuentran de inmediato, por lo que representa el trabajo mínimo. Respuesta esperada: existe camino.

Peor caso —grafo bipartito completo desbalanceado $K_{a,b}$. Con partes de tamaños a y b tales que $|a - b| \geq 2$, este grafo *probablemente no admite* camino hamiltoniano: en un grafo bipartito, un camino debe alternar entre ambas partes, así que sus tamaños pueden diferir en a lo sumo 1. Al no haber solución, los algoritmos exactos se ven obligados a agotar el espacio de búsqueda antes de poder concluir que no existe, lo que materializa su comportamiento de peor caso. Concretamente, tomamos $a = \min(\lceil n/2 \rceil + 1, n)$ y $b = n - a$.

Caso promedio —grafo aleatorio $G(n, p)$. Un grafo de Erdős-Rényi con probabilidad de arista $p = 0,5$, en el que cada posible arista se incluye de forma independiente con esa probabilidad. Su respuesta es variable y representa el comportamiento *típico* del algoritmo sobre entradas sin estructura especial.

Rangos de tamaño y número de instancias. Para cada combinación de *algoritmo* y *tipo de caso*, hicimos crecer el tamaño desde $n = 4$ y repetimos cada configuración **5** veces (réplicas), generando así 5 instancias por punto. En el caso promedio esas cinco instancias son grafos aleatorios distintos —lo que ayuda a reducir el sesgo de una instancia particular—, mientras que en el mejor y el peor caso, al ser deterministas, las réplicas miden sobre todo la variabilidad temporal de la medición. La generación es **reproducible** porque todo el azar (tanto el de los grafos aleatorios como el del algoritmo de Monte Carlo) se deriva de una **semilla global fija** ($\text{seed} = 42$): quien repita el experimento obtendrá exactamente las mismas instancias.

Escalamiento hasta el límite computacional. Como el problema es NP-completo, cada algoritmo deja de ser tratable a un tamaño distinto. En lugar de imponer un tope único, dejamos que el propio experimento escalara “hasta donde fuera computacionalmente factible” mediante dos mecanismos: un tope de tamaño por algoritmo (salvaguarda frente al agotamiento de memoria, sobre todo en la programación dinámica) y un *corte por presupuesto de tiempo*, que interrumpe el crecimiento de n para una configuración en cuanto una corrida de prueba supera un umbral (fijado en 5 segundos). Como consecuencia, el tamaño máximo alcanzado depende del algoritmo y del tipo de caso, tal como resume la Tabla 2.

Tabla 2: Tamaño máximo de entrada n alcanzado por cada algoritmo y tipo de caso (desde $n = 4$), con 5 réplicas por tamaño. Los topes reflejan el punto donde cada enfoque dejó de ser computacionalmente factible.

Algoritmo	Mejor	Peor	Promedio
Fuerza bruta	11	10	11
Backtracking	22	12	22
Procedimiento de Rubin	22	12	22
Programación dinámica (Held-Karp)	17	19	18
Monte Carlo	26	26	26

En total, el barrido produjo **1155** corridas individuales (una fila de datos brutos por réplica), que constituyen la base de todo el análisis posterior. La marcada diferencia entre los tamaños máximos —la fuerza bruta apenas supera $n = 10$, mientras que Monte Carlo llega a $n = 26$ — es en sí misma un primer resultado: anticipa el muro de complejidad que se estudia en detalle más adelante.

2.4. Entorno experimental

Todas las mediciones se tomaron en una única máquina local, ejecutando el experimento desde **Visual Studio Code**. Registrar el entorno de forma precisa es indispensable para la reproducibilidad, pues los tiempos absolutos dependen del hardware y del software subyacentes. Por eso el propio notebook captura esta información de manera automática —mediante la librería `psutil` y el módulo `platform` de Python— y la guarda en el archivo `entorno.json`, que se incluye como anexo. La Tabla 3 reproduce esos datos.

Tabla 3: Entorno de hardware y software en el que se tomaron las mediciones, registrado automáticamente en `entorno.json`.

Parámetro	Valor
<i>Hardware</i>	
Procesador	AMD64 Family 23 Model 104 Stepping 1 (AuthenticAMD)
Arquitectura	AMD64
Núcleos	4 físicos / 8 lógicos
Memoria RAM	7,33 GB
<i>Software</i>	
Sistema operativo	Windows 11
Python	3.13.3
Entorno de ejecución	Visual Studio Code (Jupyter)
<code>networkx</code>	3.6.1
<code>numpy</code>	2.3.1
<code>pandas</code>	3.0.3
<code>matplotlib</code>	3.10.9

La elección de un entorno **local y dedicado**, en lugar de una plataforma en la nube como Google Colab, fue una decisión metodológica deliberada. Aunque el notebook está diseñado para correr en ambos entornos, un servicio en la nube asigna una CPU compartida y variable, lo que introduce ruido en la medición de tiempos y compromete la comparabilidad entre corridas; además, impone límites de sesión que interrumpirían los barridos más largos. Ejecutar localmente nos dio una CPU estable, sin cortes por inactividad, y un control total del hardware que documentamos.

La **reproducibilidad** del experimento se sustenta en tres pilares. Primero, una **semilla global fija** (`seed = 42`) de la que se deriva todo el azar, de modo que las instancias generadas y el comportamiento del algoritmo de Monte Carlo son idénticos en cada ejecución. Segundo, el **registro automático del entorno** en `entorno.json`, que deja constancia exacta de las versiones de software empleadas. Tercero, el **almacenamiento de los datos brutos** en un archivo CSV: cualquier persona puede volver a generar las mismas gráficas y estadísticas a partir de ellos, sin necesidad de repetir el costoso barrido experimental. En la siguiente subsección detallamos el procedimiento paso a paso para replicar el estudio.

2.5. Procedimiento de experimentación

El experimento se organiza en dos *notebooks* de Jupyter que reflejan las dos fases del trabajo y que, deliberadamente, **desacoplan la generación de datos de su análisis**. Esta separación es importante: la generación de datos es costosa y conviene ejecutarla una sola vez en un entorno local dedicado, mientras que el análisis es liviano y cualquier persona puede repetirlo a partir de los datos ya guardados, sin rehacer el barrido. La estructura de archivos es la siguiente:

```
proyecto-camino-hamiltoniano/
|-- notebooks/
|   |-- Proyecto_Final_ADA.ipynb      (Fase 1: implementaciones y pruebas)
|   |-- Resultados_Experimento.ipynb  (Fase 2: experimento y analisis)
|-- resultados/
|   |-- resultados_crudos.csv          (una fila por corrida)
|   |-- resumen_estadistico.csv        (agregados por algoritmo/caso/tamano)
|   |-- entorno.json                  (registro del hardware y software)
|-- figuras/                          (las 14 figuras generadas)
|-- informe/
|   |-- informe_final.pdf
|-- requirements.txt
'-- README.md
```

Para replicar el experimento por completo basta con seguir estos pasos:

1. **Requisitos previos.** Instalar Python 3.13 (o una versión compatible) y las dependencias del proyecto, listadas en `requirements.txt`: `networkx`, `pandas`, `matplotlib`, `psutil` y, para las pruebas unitarias, `pytest` e `ipytest`. Se instalan con `pip install -r requirements.txt`.
2. **Validación de la corrección (Fase 1).** Abrir `Proyecto_Final_ADA.ipynb` y ejecutarlo por completo. Este notebook contiene las implementaciones de los cinco algoritmos, sus pruebas de ejecución sobre un grafo de demostración y una batería de pruebas unitarias con `pytest`; que todas pasen confirma que las implementaciones son correctas antes de medir su rendimiento.
3. **Generación de los datos (Fase 2).** Abrir `Resultados_Experimento.ipynb` en Visual Studio Code y ejecutar las celdas en orden. El notebook, en secuencia: (a) registra el entorno de ejecución en `entorno.json`; (b) define la infraestructura de medición (la clase `Timer`, el envoltorio `GrafoContador` y la medición con `tracemalloc`); y (c) lanza el barrido experimental, que recorre el producto $\text{algoritmo} \times \text{tipo de caso} \times \text{tamaño}$ con las réplicas configuradas y vuelca los resultados brutos en `resultados/resultados_crudos.csv`. Esta última celda es la *etapa costosa* del proceso.
4. **Análisis y figuras.** Las celdas finales del mismo notebook cargan el CSV bruto, calculan el resumen estadístico (`resumen_estadistico.csv`) y generan las 14 figuras, que se guardan automáticamente en la carpeta `figuras/` con nombres descriptivos.

La configuración del barrido es la que reportamos a lo largo de esta sección: semilla global `seed = 42`, **5 réplicas** por configuración, un **corte por presupuesto de tiempo** de 5 segundos y topes de tamaño por algoritmo. Estos parámetros están centralizados en una única celda de configuración, de modo que quien quiera un barrido más exhaustivo (o más rápido) solo debe modificar allí los valores. Gracias a la semilla fija, una nueva ejecución producirá exactamente las mismas instancias y decisiones aleatorias; solo los tiempos absolutos variarán, como es natural, según el hardware donde se corra.

3. Resultados

A partir de las 1155 corridas individuales del barrido experimental —una por cada réplica de cada configuración— calculamos con Pandas un resumen estadístico de 231 filas, agrupando por algoritmo, tipo de caso y tamaño, con las medidas de tendencia central (media, mediana y moda) y de dispersión (desviación estándar, mínimo y máximo). En esta sección presentamos ese material ya procesado: las tablas comparativas y las figuras que sintetizan el comportamiento observado. La interpretación de estos resultados se desarrolla en la sección siguiente.

3.1. Comparación del número de operaciones básicas

Como el número de operaciones básicas (consultas de adyacencia) es *determinista* —depende solo de la instancia, no del hardware—, permite una comparación limpia entre los cinco algoritmos. La Tabla 4 reporta la moda de las operaciones en un tamaño común a todos, $n = 10$, para los tres tipos de caso.

Tabla 4: Número (moda) de operaciones básicas en $n = 10$, por algoritmo y tipo de caso. El número de aristas es $m = 45$ (mejor), $m = 24$ (peor) y $m = 22$ (promedio).

Algoritmo	Mejor	Peor	Promedio
Fuerza bruta	9	7 568 640	738
Backtracking	81	251 040	40
Procedimiento de Rubin	1 053	450 720	562
Programación dinámica (Held–Karp)	46 080	10 080	9 143
Monte Carlo	81	84 780	40

Los contrastes son notables. En el **mejor caso**, la fuerza bruta resuelve con apenas 9 operaciones (la primera permutación ya es un camino), mientras que la programación dinámica realiza 46 080 sin importar lo “fácil” que sea la instancia. En el **peor caso** la situación se invierte por completo: la fuerza bruta se dispara a más de 7,5 millones de operaciones, en tanto que la programación dinámica se mantiene en 10 080 —la menor de todas, por ser insensible a la instancia—. También en el peor caso se observa que el procedimiento de Rubin realiza más operaciones que el *backtracking* simple (450 720 frente a 251 040), pese a explorar menos nodos, debido al costo de su test de admisibilidad.

3.2. Tiempo de ejecución frente al tamaño de entrada

La Figura 1 muestra el tiempo medio de ejecución frente al tamaño n , en escala logarítmica, para los tres tipos de caso. La curva de la fuerza bruta es la primera en elevarse, seguida por el *backtracking* y Rubin; la programación dinámica crece de forma más suave, y Monte Carlo se mantiene prácticamente plano en la parte inferior de cada gráfica.

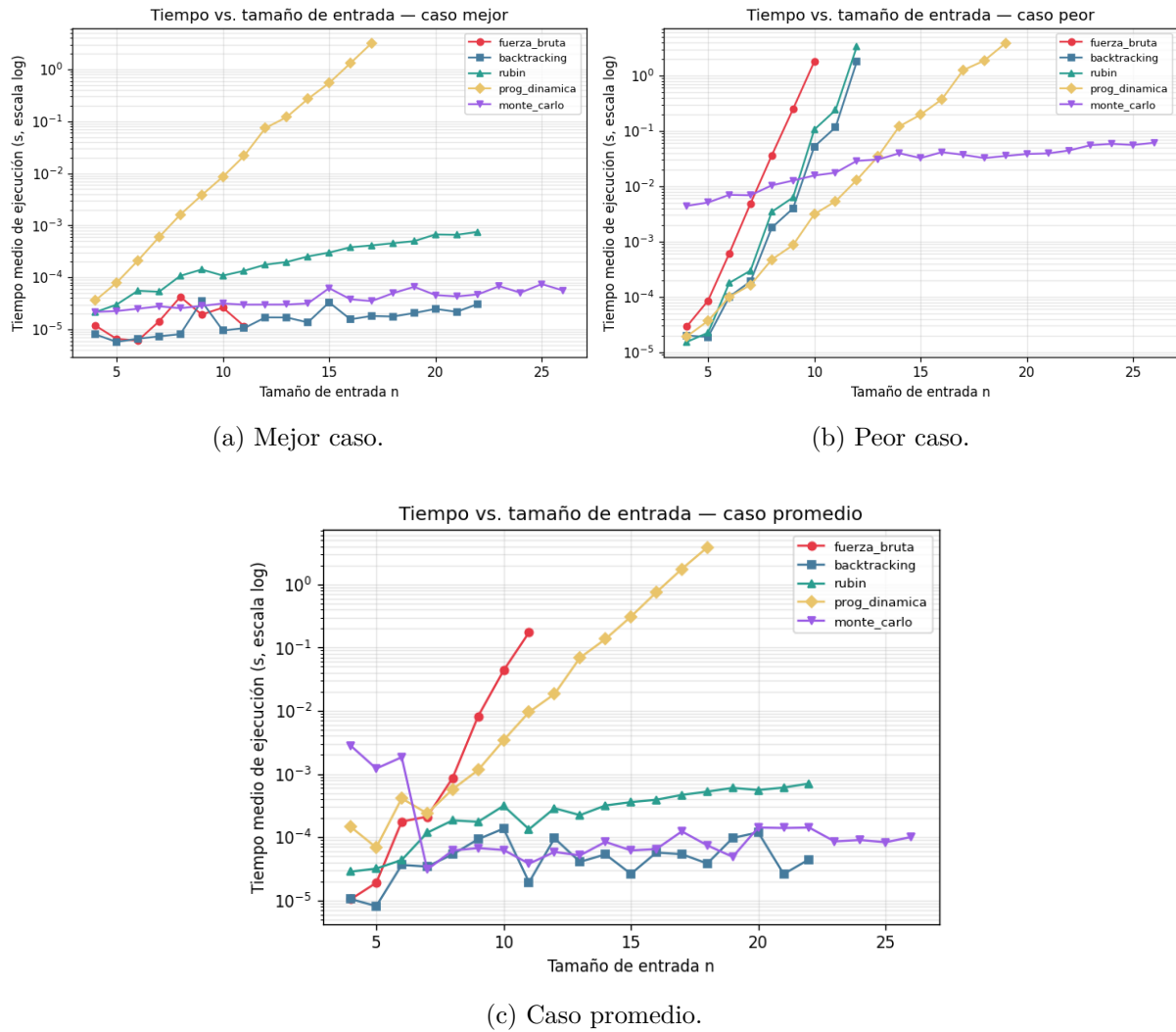


Figura 1: Tiempo medio de ejecución (escala logarítmica) frente al tamaño de entrada n , para cada tipo de caso.

El tamaño hasta el que fue posible medir cada algoritmo lo determinó un corte automático por presupuesto de tiempo, fijado en 5 segundos sobre una corrida de sondeo. La Tabla 5 registra las configuraciones donde se activó ese corte y el tiempo de la corrida que lo disparó; en cada caso, el mayor tamaño efectivamente medido es el inmediatamente anterior.

Tabla 5: Configuraciones en las que se activó el corte por presupuesto de tiempo (5 s) y tiempo de la corrida de sondeo que lo disparó.

Algoritmo	Tipo de caso	Tiempo del sondeo
Fuerza bruta	Peor	18,8 s (en $n = 11$)
Backtracking	Peor	5,4 s (en $n = 13$)
Procedimiento de Rubin	Peor	7,7 s (en $n = 13$)
Programación dinámica (Held–Karp)	Mejor	6,9 s (en $n = 18$)
Programación dinámica (Held–Karp)	Peor	9,4 s (en $n = 20$)
Programación dinámica (Held–Karp)	Promedio	9,5 s (en $n = 19$)

3.3. Variabilidad entre réplicas

Para valorar la estabilidad de las mediciones de tiempo, la Figura 2 presenta los diagramas de caja del tiempo entre las cinco réplicas de cada tamaño, en el caso promedio. En general las cajas son estrechas, lo que indica mediciones consistentes; los valores atípicos aislados que aparecen en algunos tamaños pequeños corresponden a tiempos del orden de los microsegundos, donde el ruido del sistema operativo es proporcionalmente mayor.

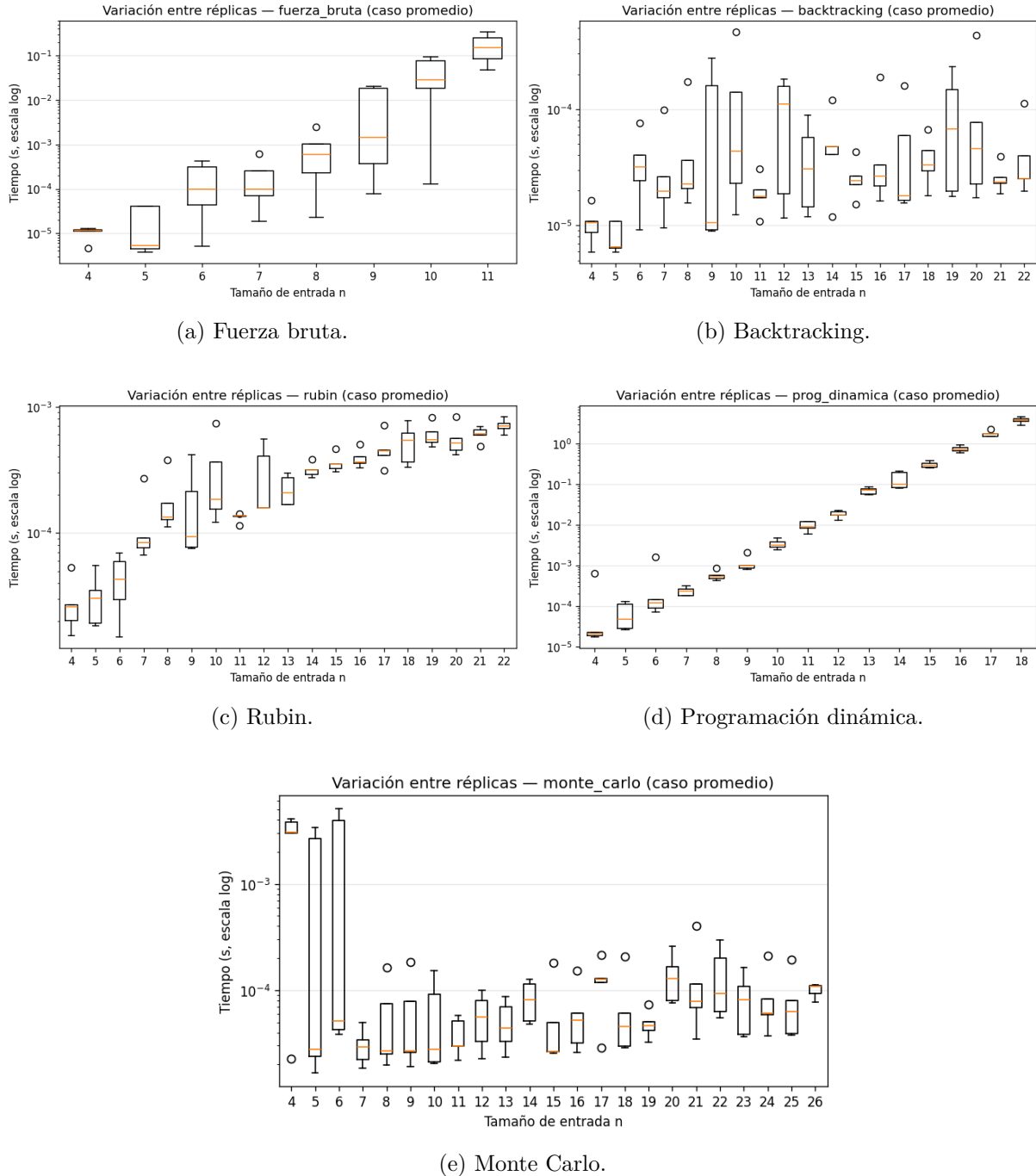


Figura 2: Variación del tiempo de ejecución entre las cinco réplicas de cada tamaño (caso promedio, escala logarítmica).

La Figura 3 contrasta el pico de memoria frente a n (escala logarítmica). El comportamiento separa nítidamente a la programación dinámica del resto: mientras los cuatro enfoques de espacio

polinómico se mantienen en unos pocos kilobytes —del orden de 1 a 5 kB en los tamaños medidos—, la programación dinámica crece de forma exponencial, alcanzando ya cientos de kilobytes en tamaños moderados (del orden de 286 kB en $n = 10$).

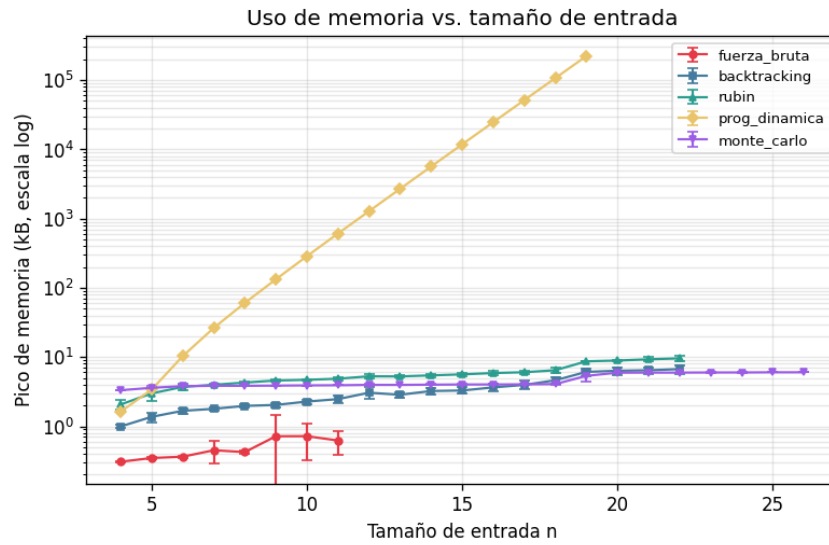
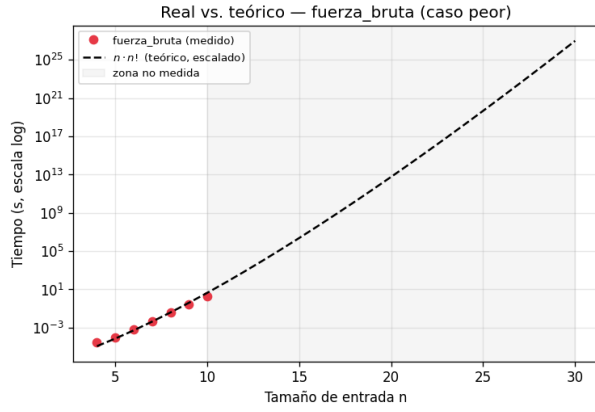


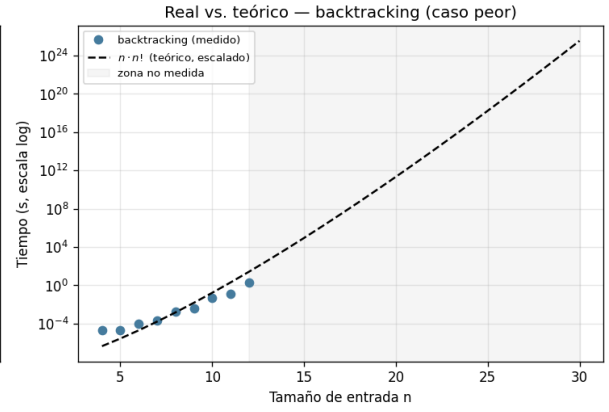
Figura 3: Pico de memoria frente al tamaño de entrada n (escala logarítmica). Los bigotes representan la desviación estándar.

3.4. Contraste con la complejidad teórica

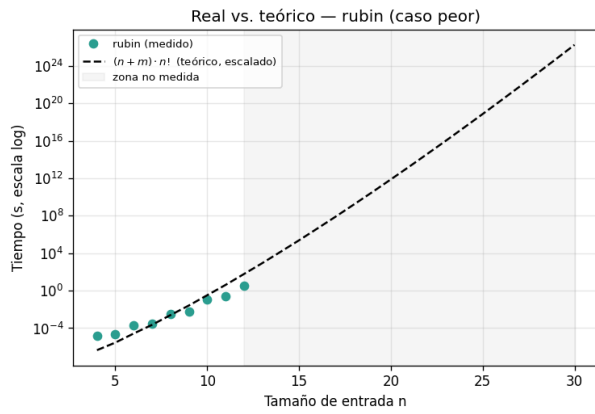
Por último, la Figura 4 superpone el tiempo medido de cada algoritmo (caso peor) con su cota de complejidad teórica, ajustada por un único factor de escala y prolongada más allá del rango medido para exhibir su comportamiento asintótico. Los puntos medidos siguen de cerca la curva teórica en la región disponible, lo que permite proyectar el crecimiento esperado en tamaños que la máquina no alcanzó a medir.



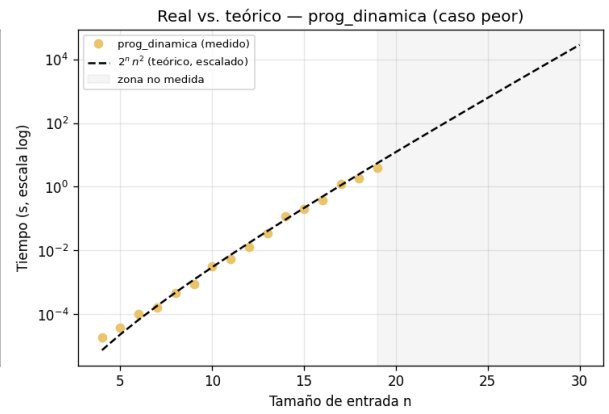
(a) Fuerza bruta.



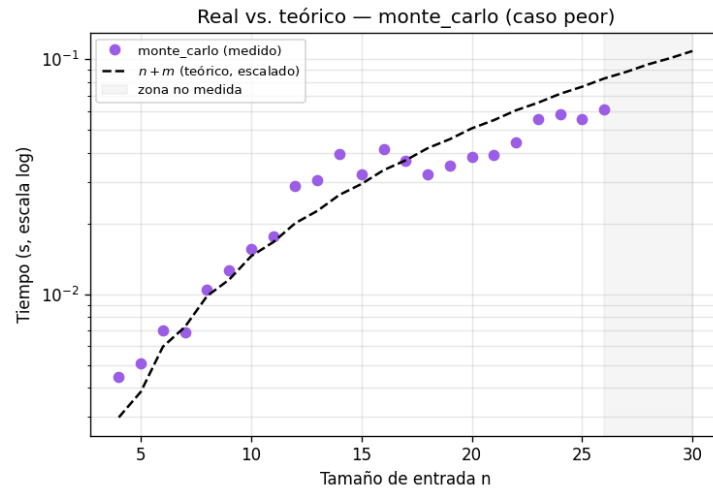
(b) Backtracking.



(c) Rubin.



(d) Programación dinámica.



(e) Monte Carlo.

Figura 4: Tiempo medido (caso peor) frente a la complejidad teórica de cada algoritmo, con la curva teórica escalada y extrapolada hasta $n = 30$ (zona sombreada, no medida).

4. Análisis y Discusión

4.1. Interpretación de los patrones observados

El resultado más general del experimento es la confirmación empírica del **muro de complejidad** inherente a un problema NP-completo: todos los enfoques exactos dejan de ser tratables con tamaños de entrada sorprendentemente pequeños. La fuerza bruta fue la primera en chocar con él, incapaz de superar $n = 11$ en el peor caso (una sola instancia le tomó 18,8 segundos, según la Tabla 5), un comportamiento del todo coherente con su complejidad $O(n \cdot n!)$: el factorial crece tan rápido que unos pocos vértices adicionales multiplican el trabajo por órdenes de magnitud. El contraste con la teoría, resumido en la Figura 4, es satisfactorio: en la región medida los puntos siguen de cerca las curvas teóricas escaladas, lo que nos da confianza para proyectar, mediante la extrapolación, el crecimiento que la máquina no alcanzó a medir.

El segundo patrón claro es la **sensibilidad al tipo de instancia**, que separa a los algoritmos en dos familias. Los de búsqueda —fuerza bruta, *backtracking* y Rubin— son extremadamente sensibles: en el mejor caso (grafo completo) la fuerza bruta resuelve con apenas 9 operaciones (Tabla 4), porque la primera permutación ya es un camino válido; pero en el peor caso, al no existir solución, se ven obligados a agotar el espacio de búsqueda y su costo se dispara a millones de operaciones. Esta sensibilidad explica por qué el *backtracking* y Rubin, gracias a sus podas, alcanzan $n = 22$ en los casos mejor y promedio pero se estancan en $n = 12$ en el peor: la poda es efficacísima cuando hay estructura que aprovechar o una solución que hallar pronto, pero pierde casi toda su fuerza cuando el algoritmo debe *demostrar la inexistencia* de un camino. La programación dinámica, en cambio, pertenece a la otra familia: es **insensible a la instancia**. Su conteo de operaciones se mantiene en el mismo orden de magnitud en los tres casos (46 080, 10 080 y 9 143), porque siempre construye la tabla completa de $2^n \cdot n$ estados sin importar la dificultad de la entrada. Por eso alcanza tamaños parecidos (17 a 19) en los tres tipos de caso.

4.2. Resultados inesperados

Tres observaciones desafiaron nuestra intuición inicial y merecen una explicación detenida.

El primero, y más llamativo, es una **inversión en el peor caso**: allí la programación dinámica —el enfoque de memoria más costosa— resultó ser el *más eficiente en operaciones* de todos los métodos exactos, con solo 10 080 operaciones frente a los más de 7,5 millones de la fuerza bruta e incluso los 251 040 del *backtracking* (Tabla 4). La explicación es que el costo de la programación dinámica, $O(2^n n^2)$, aunque exponencial, crece mucho más despacio que el factorial $O(n!)$ que los métodos de búsqueda deben recorrer cuando no hay camino: para $n = 10$, 2^n está en el orden de 10^3 mientras que $n!$ ronda los $3,6 \times 10^6$. Dicho de otro modo, en las instancias verdaderamente difíciles conviene comprometer la memoria de la programación dinámica a cambio de no explorar el gigantesco árbol combinatorio de la búsqueda.

El segundo resultado inesperado es que el **procedimiento de Rubin realizó más operaciones que el *backtracking* simple** en el peor caso (450 720 frente a 251 040) y no logró superar el mismo tamaño máximo ($n = 12$). Esto parece contradecir la idea de que sus reglas de deducción “mejoran” la búsqueda. La explicación está en la métrica y en la instancia: nuestra operación básica es la consulta de adyacencia, que contabiliza el *trabajo primitivo total*, incluida la sobrecarga del test de admisibilidad; Rubin explora menos nodos, pero cada nodo le cuesta muchas más consultas de adyacencia (las que gastan las reglas F1, F5 y F7). En el peor caso bipartito, esas reglas no podan lo suficiente como para compensar su propio costo. Es plausible que en grafos con otra estructura —más dispersos o con vértices de grado bajo— la poda de Rubin sí rinda; nuestro peor caso, por su regularidad, resulta ser un escenario particularmente desfavorable para él.

El tercero tiene que ver con **qué recurso limitó realmente a la programación dinámica**. Anticipábamos que sería la memoria, dado su costo espacial $O(2^n n)$; sin embargo, dentro de nuestro presupuesto de 5 segundos, fue el *tiempo* el que la detuvo primero (por ejemplo, en el peor caso se cortó en $n = 20$ tras 9,4 s, según la Tabla 5). En una máquina con 7,33 GB de RAM, la tabla para $n = 20$ todavía cabe holgadamente en memoria; habría que llevar n bastante más allá (aproximadamente hasta $n \approx 24$ –26) para que el consumo exponencial de memoria se volviera el cuello de botella. Así, cuál es el recurso limitante no es una propiedad absoluta del algoritmo, sino que depende del presupuesto de tiempo y del hardware concretos.

En cuanto al algoritmo de **Monte Carlo**, se comportó como esperábamos de un método probabilístico: fue el único que escaló hasta $n = 26$ (limitado por nuestro tope, no por su costo) y el más económico por intento en los casos mejor y promedio, empatando con el *backtracking* en apenas 40 operaciones en el caso promedio. Conviene subrayar que, en nuestras instancias, *no* observamos falsos negativos: en el peor caso respondió correctamente que no hay camino (su error de un solo lado se lo garantiza) y en los casos con solución siempre lo encontró. No obstante, esto se debe en buena parte a que el caso promedio usó grafos densos ($p = 0,5$), donde la probabilidad de éxito por intento es alta; su verdadero talón de Aquiles —los grafos poco densos— no llegó a ponerse a prueba, como discutimos enseguida.

4.3. Limitaciones y amenazas a la validez

Nuestro estudio tiene limitaciones que conviene declarar con honestidad. La más importante es consecuencia directa de la NP-completitud del problema: los **tamaños de entrada alcanzables son muy pequeños** (a lo sumo $n = 26$, y bastante menos para los métodos exactos), de modo que nuestras afirmaciones sobre el comportamiento asintótico se estriban en la extrapolación de las curvas teóricas más que en mediciones directas en tamaños grandes.

En segundo lugar, el **ruido en la medición del tiempo** es inevitable: aunque cronometraremos en corridas limpias y repetimos cada configuración cinco veces, en los tamaños pequeños los tiempos son del orden de microsegundos, donde la sobrecarga del sistema operativo, el recolector de basura y el propio intérprete de Python introducen variabilidad; los valores atípicos aislados que se ven en los boxplots de la Figura 2 son manifestación de ese ruido. Un número de réplicas mayor daría estimaciones más estables.

En tercer lugar, nuestras decisiones metodológicas condicionan lo que observamos. El **peor caso** se representó con una única familia de grafos (bipartitos completos desbalanceados); es un peor caso legítimo por carecer de solución, pero no necesariamente el que más estresa a *cada* algoritmo. Asimismo, la **operación básica** elegida —la consulta de adyacencia— retrata a Rubin de forma desfavorable; contar en cambio las llamadas recursivas lo mostraría bajo otra luz. Por último, la elección de $p = 0,5$ para el caso promedio produjo grafos densos que no exponen la debilidad de Monte Carlo frente a las instancias dispersas. Todas estas son fuentes de sesgo que acotan el alcance de nuestras conclusiones.

4.4. Preguntas abiertas y trabajo futuro

Los resultados anteriores sugieren varias líneas para experimentos futuros. La primera, natural, es **someter a Monte Carlo a grafos dispersos** (con p pequeño y estructuras que tengan pocos caminos hamiltonianos) para cuantificar su tasa de falsos negativos y estudiar cómo la probabilidad de éxito por intento depende de la densidad. La segunda es **variar la familia del peor caso** para identificar en qué tipo de instancias las reglas de deducción de Rubin sí compensan su sobrecarga. La tercera es **separar experimentalmente los límites de tiempo y de memoria** de la programación dinámica, ampliando el presupuesto de tiempo y midiendo

hasta que sea la memoria quien la detenga, quizás con una implementación optimizada en espacio. Finalmente, sería valioso incorporar algoritmos exactos más avanzados —como el método algebraico de inclusión-exclusión de Björklund, que rompe la barrera del 2^n — para observar hasta dónde es posible empujar la resolución exacta del problema.

5. Conclusiones

A partir del estudio experimental de cinco enfoques para el problema del Camino Hamiltoniano, recogemos los siguientes hallazgos y aprendizajes:

1. **La NP-completitud se siente en la práctica, y muy pronto.** Los cinco algoritmos dejaron de ser tratables con tamaños de entrada sorprendentemente pequeños ($n \leq 26$, y mucho menos para los métodos exactos). El experimento hace tangible que, para un problema NP-completo, no es cuestión de esperar un poco más o conseguir una máquina mejor: unos pocos vértices adicionales bastan para volver imposible el cálculo.
2. **No existe un “mejor algoritmo” absoluto; la elección depende de la instancia y del recurso crítico.** En instancias fáciles o con solución cercana, el *backtracking* y Rubin son imbatibles gracias a la poda; en las instancias difíciles (sin solución), la programación dinámica es preferible porque su costo exponencial $O(2^n n^2)$ crece más despacio que el factorial de la búsqueda; y cuando basta una respuesta probable a bajo costo, Monte Carlo escala más que todos. La comparación empírica reveló matices que la sola complejidad teórica no anticipaba, como la inversión de eficiencia de la programación dinámica en el peor caso.
3. **El intercambio espacio–tiempo es real y medible.** La programación dinámica compra velocidad con memoria: fue la más resistente en tiempo entre los exactos, pero su consumo exponencial de memoria la condena a tamaños moderados. Cuál de los dos recursos —tiempo o espacio— termina siendo el cuello de botella no es una propiedad fija del algoritmo, sino que depende del presupuesto y del hardware concretos.
4. **La aleatorización cambia las reglas del juego.** Monte Carlo mostró que renunciar a la exactitud —aceptando un error de un solo lado— permite abordar tamaños fuera del alcance de los métodos exactos, con un costo polinómico por intento. El precio es una garantía más débil, cuya solidez depende de la estructura de la entrada.
5. **Un marco experimental riguroso es tan importante como los algoritmos.** Aprendimos que medir bien exige cuidado: cronometrar en corridas limpias y separadas de la instrumentación, fijar una semilla para la reproducibilidad, repetir cada configuración, documentar el entorno y almacenar los datos brutos. Desacoplar la generación de datos de su análisis resultó clave para poder repetir y refinar el estudio sin rehacer el costoso barrido.
6. **Impacto del uso de la inteligencia artificial.** Desarrollamos el proyecto con el apoyo de un asistente de IA, y el balance fue muy positivo, siempre bajo nuestra supervisión y criterio. La IA aceleró tareas concretas —proponer implementaciones, depurar el código, estructurar el diseño experimental y redactar de forma ordenada—, pero su aporte más valioso fue como interlocutor técnico: nos ayudó a anticipar problemas antes de que ocurrieran (por ejemplo, a dimensionar el costo del barrido y a evitar una configuración experimental que habría tardado horas), a discutir decisiones de diseño y a mantener la coherencia entre el notebook y el informe. La responsabilidad de validar cada resultado, aprobar cada decisión y comprender a fondo el contenido siguió siendo enteramente nuestra; la IA fue una herramienta de productividad y de aprendizaje, no un sustituto del trabajo intelectual.

En síntesis, el problema del Camino Hamiltoniano resultó un excelente laboratorio para observar, con datos propios, la frontera entre lo computacionalmente factible y lo intratable, y para comprender que enfrentar un problema NP-completo consiste, en el fondo, en elegir sabiamente qué sacrificar: tiempo, memoria o certeza.

6. Referencias

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press.
- Sipser, M. (2013). *Introduction to the Theory of Computation* (3rd ed.). Cengage Learning.
- Rubin, F. (1974). A search procedure for Hamilton paths and circuits. *Journal of the ACM (JACM)*, 21(4), 576-580.
- Real Python. *Python Context Managers and the "with" Statement*. Recuperado de <https://realpython.com/python-with-statement/>.
- NetworkX Developer Team. *NetworkX Reference Documentation*. Recuperado de <https://networkx.org/>.

7. Anexos

Notebooks del Proyecto

- **FASE 1 — Proyecto_Final_ADA.ipynb**: Contiene las implementaciones definitivas de los cinco algoritmos analizados, las suites de pruebas unitarias integradas y la verificación lógica inicial de corrección algorítmica.
- **FASE 2 — Resultados_Experimento.ipynb**: Contiene el motor y arnés experimental, los decoradores de instrumentación y el pipeline de graficación estadística con el que se generaron las figuras y tablas comparativas.

Conjuntos de Datos

- **resultados_crudos.csv**: Archivo estructurado plano conteniendo las 1155 corridas completas con el detalle analítico por réplica y variables experimentales.
- **resumen_estadistico.csv**: Consolidado agrupado mediante Pandas que resume las métricas de tendencia central y dispersión presentadas a lo largo de este informe.
- **entorno.json**: Archivo de metadatos con el perfil técnico de hardware y versiones de software del entorno de ejecución local.